

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

CHINMAY G HEGDE (1BM24CS083)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **CHINMAY G HEGDE (1BM24CS083)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write a program to obtain the Topological ordering of vertices in a given digraph.	1
2	Implement Johnson Trotter algorithm to generate permutations.	4
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	6
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	8
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	10
6	Implement 0/1 Knapsack problem using dynamic programming.	12
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	15
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	18
9	Implement Fractional Knapsack using Greedy technique.	23
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	25
11	Implement "N-Queens Problem" using Backtracking.	27

Course outcomes:

CO1	Analyze time complexity of recursive and non-recursive algorithms using asymptotic notations
CO2	Apply various algorithm design techniques for the given problem.
CO3	Apply the knowledge of complexity classes P, NP, and NP-Complete and prove certain problems are NP-Complete
CO4	Design efficient algorithms and conduct practical experiments to solve problems.

I N D E X

NAME: CHINMAY HECHDE STD.: 18M24CS083 SEC.: 44 ROLL NO.: SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	2/03/2026	Lecture 11 and 268		
2	2/03/2026	Topological Sorting		
3	9/03/2026	Johnson Trotter permutation		
4	16/03/2026	① GCD using euclid algorithm ② Fibonacci ③ Tower of Hanoi		
5	23/03/2026	④ Selection Sort ⑤ Bubble Sort		
6	28/03/2026	Merge Sort		
7	28/03/2026	Quick Sort		
8	13/04/2026	Fractional Knapsack (greedy)		
9	27/04/2026	0/1 Knapsack Dynamic Programming		
10	27/04/2026	LeetCode 801		
11	4/05/2026	Prim's Algorithm		
12	4/05/2026	Kruskal's Algorithm		
13	11/05/2026	Heap Sort		
14	11/05/2026	① Floyd's Algorithm ② LeetCode 287		
15	11/05/2026	Dijkstra's Algorithm		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
16	11/05/26	Knapsack greedy		
17	25/05/26	N-Queens Problem		

Lab program 1:

Write a program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>
#define MAX 10

int adj[MAX][MAX], vis[MAX], stack[MAX];
int top = -1; int n;

void dfs(int v) {
    vis[v] = 1;
    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && vis[i] == 0) {
            dfs(i);
        }
    }
    stack[++top] = v;
}

void topologicalSort() {
    for (int i = 0; i < n; i++) {
        if (vis[i] == 0) {
            dfs(i);
        }
    }

    printf("Topological Order: ");
    while (top != -1) {
        printf("%d ", stack[top--]);
    }
}

void main() {
    int i, j;
    printf("Enter No. of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
        vis[i] = 0;
    }

    topologicalSort();
    printf("\n");
}
```

Output:

```
Enter No. of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 0
0 0 0 0
Topological Order: 0 2 1 3
```

Leetcode 207 Course Scheduling using Topological Sorting

Code:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int n = numCourses;
    int p = prerequisitesSize;
    int queue[2000];
    int front = 0, rear = 0;

    int adj[n][n];
    int indegree[n];
    for(int i = 0; i < n; i++){
        indegree[i] = 0;
        for(int j = 0; j < n; j++){
            adj[i][j] = 0;
        }
    }

    for(int i = 0; i < p; i++){
        int from = prerequisites[i][1];
        int to = prerequisites[i][0];

        if (to == from) return false;
        if (!adj[from][to]) {
            adj[from][to] = 1;
            indegree[to]++;
        }
    }

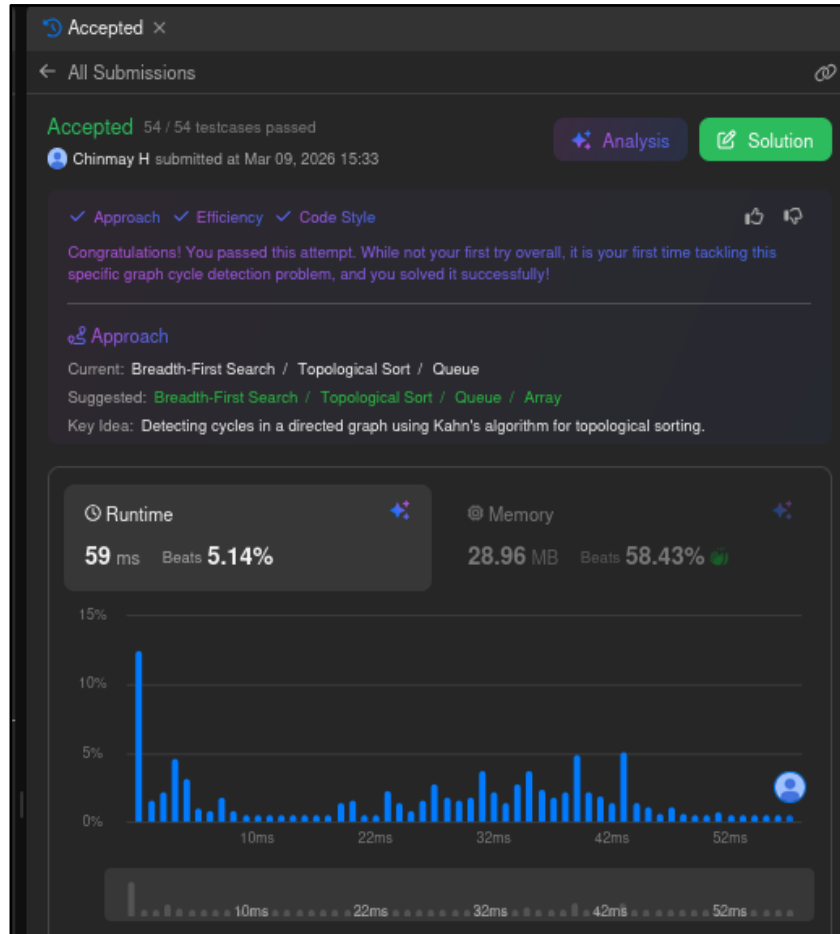
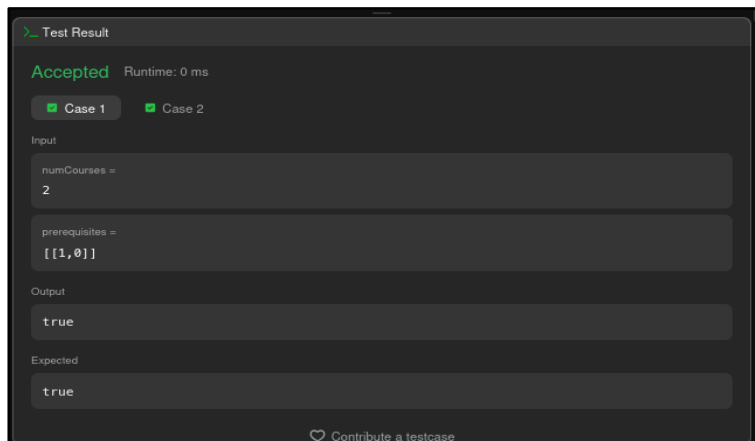
    for (int i = 0; i < n; i++){
        if (indegree[i] == 0){
            queue[rear++] = i;
        }
    }
}
```

```

int count = 0;
while(front < rear) {
    int val = queue[front++];
    count++;

    for (int i = 0; i < n; i++) {
        if (adj[val][i]) {
            indegree[i]--;
            if (indegree[i] == 0) {
                queue[rear++] = i;
            }
        }
    }
}
return count == numCourses;
}

```



Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

struct node {
    int value;
    int dir;
};

int getMobile(struct node a[], int n) {
    int mobile = -1, mobile_index = -1;

    for (int i = 0; i < n; i++) {
        if (a[i].dir == LEFT && i != 0 && a[i].value > a[i-1].value && a[i].value > mobile) {
            mobile = a[i].value;
            mobile_index = i;
        }
        if (a[i].dir == RIGHT && i != n-1 && a[i].value > a[i+1].value && a[i].value > mobile)
        {
            mobile = a[i].value;
            mobile_index = i;
        }
    }
    return mobile_index;
}

void printPermutation(struct node a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i].value);
    }
    printf("\n");
}

void johnsonTrotter(int n) {
    struct node a[n];
    for (int i = 0; i < n; i++) {
        a[i].value = i + 1;
        a[i].dir = LEFT;
    }

    printPermutation(a, n);

    while (1) {
        int mobile_index = getMobile(a, n);
```



```

        if (mobile_index == -1)
            break;

        int swap_index = mobile_index + a[mobile_index].dir;

        struct node temp = a[mobile_index];
        a[mobile_index] = a[swap_index];
        a[swap_index] = temp;

        mobile_index = swap_index;

        for (int i = 0; i < n; i++) {
            if (a[i].value > a[mobile_index].value) {
                a[i].dir = -a[i].dir;
            }
        }

        printPermutation(a, n);
    }
}

void main() {
    int n;

    printf("Enter n: ");
    scanf("%d", &n);

    johnsonTrotter(n);
    printf("\n");
}

```

Output:

```

Enter n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

```

Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int arr[], int low, int mid, int high) {
    int i = low, j = mid + 1, k = 0;
    int temp[100];

    while (i <= mid && j <= high) {
        if (arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= high)
        temp[k++] = arr[j++];

    for (i = low, k = 0; i <= high; i++, k++)
        arr[i] = temp[k];
}

void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

void main() {
    int n, i;
    int arr[100000];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("\nEnter the elements:\n");
```

```

for (i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}
clock_t start, end;
start = clock();
mergeSort(arr, 0, n - 1);
end = clock();

double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("\nSorted array:\n");
for(i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n\nTime Taken = %f ms\n", time_taken);
}

```

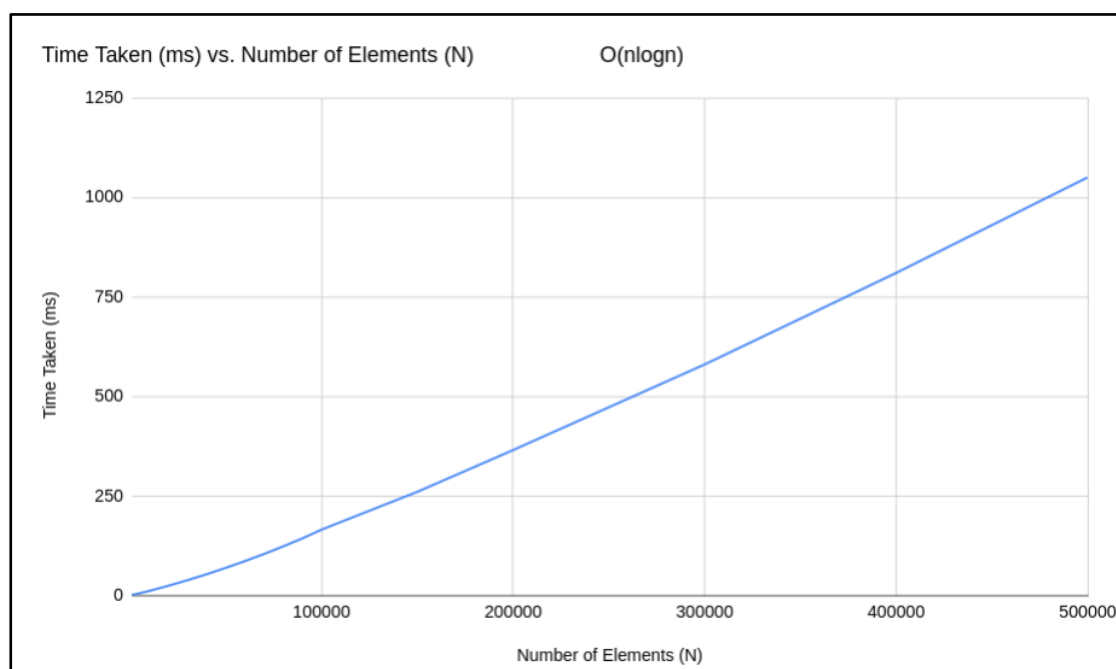
Output:

```

Enter number of elements: 5
Enter the elements:
1 6 3 9 6
Sorted array:
1 3 6 6 9
Time Taken = 0.003000 ms

```

Graph:



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

void main() {
    int n, i;
    int arr[100000];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("\nEnter the elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

```

clock_t start, end;
start = clock();
quickSort(arr, 0, n - 1);
end = clock();

double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("\nSorted array:\n");
for(i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n\nTime Taken = %f ms\n", time_taken);
}

```

Output:

```

Enter number of elements: 5

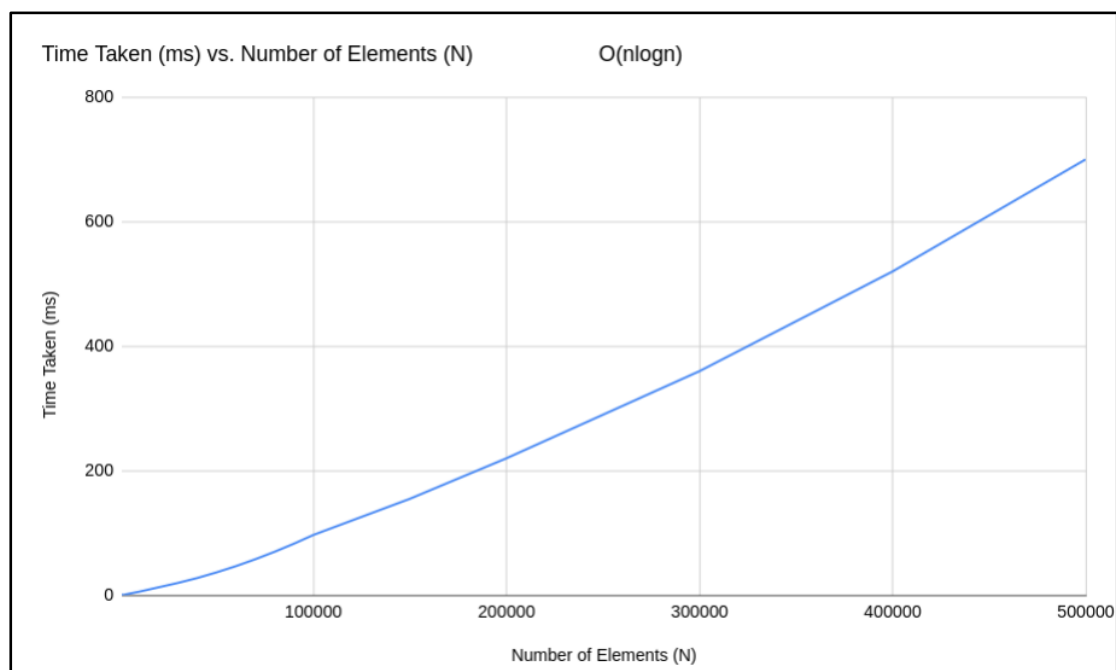
Enter the elements: 1 6 3 9 6

Sorted array:
1 3 6 6 9

Time Taken = 0.003000 ms

```

Graph:



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <time.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest]) largest = left;

    if (right < n && arr[right] > arr[largest]) largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i > 0; i--) {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

void main() {
    int n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```

int arr[n];
printf("\nEnter elements:\n");
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

start = clock();
heapSort(arr, n);
end = clock();

printf("\nSorted array:\n");
for (i = 0; i < n; i++)
    printf("%d ", arr[i]);
time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;
printf("\n\nTime taken = %f seconds\n", time_taken);
}

```

Output:

```

Enter number of elements: 5

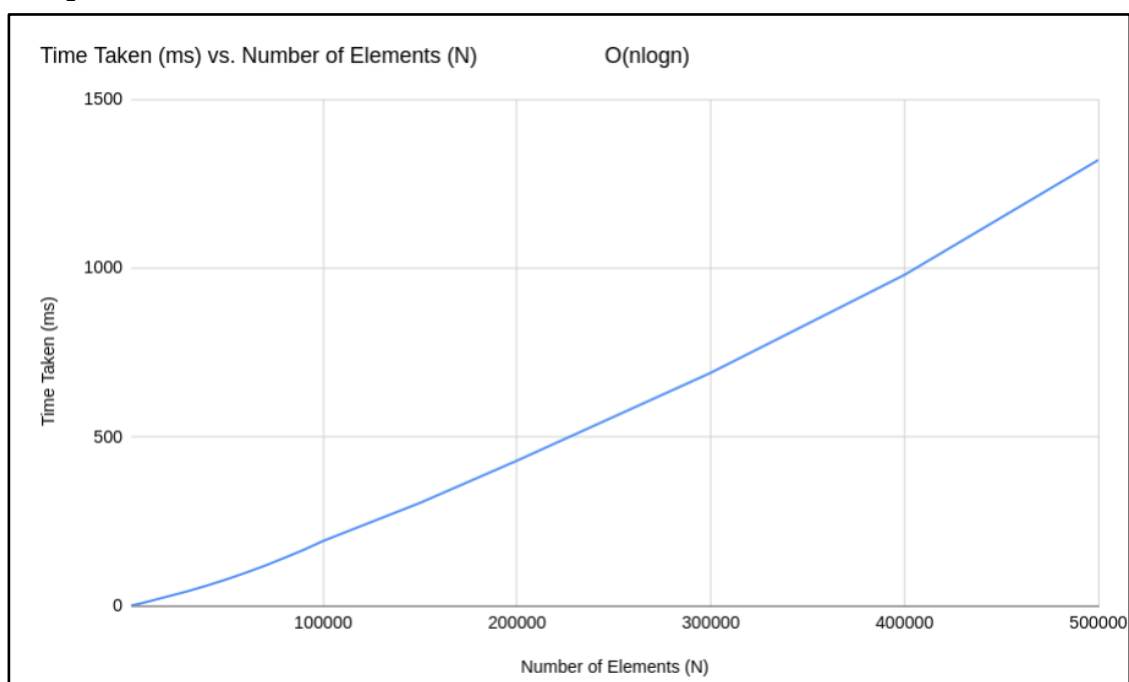
Enter elements:
45 12 89 3 27

Sorted array:
3 12 27 45 89

Time taken = 0.005000 seconds

```

Graph:



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>
#include <time.h>

int knapsack_dp(int n, int M, int W[], int P[]) {
    int table[n+1][M+1];

    for (int i = 0; i <= n; i++)
        table[i][0] = 0;

    for (int j = 0; j <= M; j++)
        table[0][j] = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= M; j++) {
            if (j < W[i-1]) { table[i][j] = table[i-1][j]; }
            else {
                int x = table[i-1][j];
                int y = P[i-1] + table[i-1][j - W[i-1]];
                table[i][j] = (x > y) ? x : y;
            }
        }
    }
    return table[n][M];
}

void main() {
    int n, M;
    printf("Enter number of objects and capacity: ");
    scanf("%d %d", &n, &M);

    int P[n], W[n];

    printf("\nEnter profit and weight:\n");
    for (int i = 0; i < n; i++)
        scanf("%d %d", &P[i], &W[i]);

    clock_t start, end;
    start = clock();
    int profit = knapsack_dp(n, M, W, P);
    end = clock();

    double time_taken = (((double) (end - start) / CLOCKS_PER_SEC)) * 1000;
    printf("\nMax Profit: %d\n", profit);

    printf("\n\nTime Taken: %f ms\n", time_taken);
}
```


Output:

```
Enter number of objects and capacity: 3 50
Enter profit and weight:
60 100
100 120
120 30
Max Profit: 120
Time Taken: 0.007000 ms
```

Leetcode 801 Minimum swaps to make sequences increasing using Knapsack dynamic programming

Code:

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;

    for (int i = 1; i < nums1Size; i++) {
        int newKeep = INT_MAX;
        int newSwap = INT_MAX;

        if (nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1]) {
            newKeep = keep;
            newSwap = swap + 1;
        }
        if (nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1]) {
            newKeep = fmin(newKeep, swap);
            newSwap = fmin(newSwap, keep + 1);
        }
        keep = newKeep;
        swap = newSwap;
    }
    return fmin(keep, swap);
}
```

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums1 =
[1,3,5,4]

nums2 =
[1,2,3,7]

Output

1

All Submissions

Accepted 117 / 117 testcases passed

Chinmay H submitted at Apr 27, 2026 15:25

Approach Efficiency Code Style

Congratulations! You passed on your first attempt with this type of problem. You're in great form today!

Approach

Current: Dynamic Programming
Suggested: **Dynamic Programming**

Key Idea: Using dynamic programming to track minimum swaps for strictly increasing sequences.

Runtime

3 ms | Beats 45.16%

Memory

16.46 MB | Beats 35.48%

Runtime (ms)	Percentage (%)
1ms	~30%
2ms	~10%
3ms	~15%
4ms	~12%
5ms	~5%
6ms	~2%

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include <stdio.h>
#include <time.h>
#define INF 999

void main() {
    int n, i, j, k;
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int dist[n][n];
    printf("\nEnter adjacency matrix: ");
    printf("(Use %d for INF)\n", INF);

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
    start = clock();
    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                if(dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    end = clock();

    printf("\nShortest Distance Matrix:\n");

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(dist[i][j] == INF) printf("INF ");
            else printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
}
```

```

    time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;
    printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:

```

Enter number of vertices: 4

Enter adjacency matrix: (Use 999 for INF)
0   5   999 10
999 0   3   999
999 999 0   1
999 999 999 0

Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Time Taken: 0.003000 ms

```

Leetcode 287 on Floyd's Algorithm

Code:

```

int findDuplicate(int* nums, int numsSize) {
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    slow = nums[0];

    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}

```

> Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =  
[1,3,4,2,2]
```

Output

```
2
```

Expected

```
2
```

Accepted 59 / 59 testcases passed

Chinmay H submitted at May 11, 2026 15:26

✓ Approach ✓ Efficiency ✓ Code Style

Congratulations! You passed on your first attempt with an optimal solution. Your mastery of this pattern is impressive!

🧠 Approach

Current: Two Pointers / Floyd's Cycle Finding Algorithm

Suggested: **Two Pointers** / **Floyd's Cycle Finding Algorithm**

Key Idea: Modeling the array as a linked list to detect cycles using Floyd's Tortoise and Hare algorithm.

Consider: Since you nailed this, how would you adapt this cycle detection logic if the duplicate number appeared three times instead of two?

🕒 Runtime

4 ms | Beats **61.73%** 🌿

Lab Program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Code:

```
#include <stdio.h>
#include <time.h>

#define INF 999

int minKey(int key[], int mstSet[], int V) {
    int min = INF, min_index;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void primMST(int V, int graph[][V]) {
    int parent[V];
    int key[V];
    int mstSet[V];

    for (int i = 0; i < V; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("\nEdge \tWeight\n");
    int total = 0;
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
}
```

```

    }
    printf("Total cost of MST = %d\n", total);
}

void main() {
    int V;
    clock_t start, end;
    double time_taken;

    printf("Enter the number of vertices: ");
    scanf("%d", &V);

    int graph[V][V];
    printf("\nEnter the adjacency matrix:\n");
    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    start = clock();
    primMST(V, graph);
    end = clock();

    time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;
    printf("\n\nTime Taken: %f ms\n");
}

```

Output:

```

Enter the number of vertices: 5

Enter the adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edge      Weight
0 - 1     2
1 - 2     3
0 - 3     6
1 - 4     5
Total cost of MST = 16

Time Taken: 0.000000 ms

```

(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 100
#define INF 999

struct Edge {
    int u, v, w;
};
struct Edge edges[MAX], result[MAX];
int parent[MAX];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int i, int j) {
    int a = find(i);
    int b = find(j);
    parent[a] = b;
}

void sortEdges(int E) {
    struct Edge temp;
    for (int i = 0; i < E - 1; i++) {
        for (int j = 0; j < E - i - 1; j++) {
            if (edges[j].w > edges[j + 1].w) {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

void main() {
    int V, E = 0;
    int cost[MAX][MAX];
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("\nEnter cost matrix (use 0 or %d for no edge):\n", INF);
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
}
```



```

for (int i = 0; i < V; i++) {
    for (int j = i + 1; j < V; j++) {
        if (cost[i][j] != 0 && cost[i][j] != INF) {
            edges[E].u = i;
            edges[E].v = j;
            edges[E].w = cost[i][j];
            E++;
        }
    }
}

for (int i = 0; i < V; i++)
    parent[i] = i;

start = clock();
sortEdges(E);
end = clock();

int i = 0, count = 0, minCost = 0;

while (count < V - 1 && i < E) {
    int u = edges[i].u;
    int v = edges[i].v;

    int setU = find(u);
    int setV = find(v);

    if (setU != setV) {
        result[count++] = edges[i];
        minCost += edges[i].w;
        unionSet(setU, setV);
    }
    i++;
}

printf("\nEdges in MST:\n");
for (i = 0; i < count; i++) {
    printf("%d - %d : %d\n", result[i].u, result[i].v, result[i].w);
}

printf("Minimum cost = %d\n", minCost);

time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;
printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:

```
Enter number of vertices: 4

Enter cost matrix (use 0 or 999 for no edge):
0  10 6  5
10 0  0 15
6  0  0  4
5  15 4  0

Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost = 19

Time Taken: 0.004000 ms
```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

Code:

```
#include <stdio.h>
#include <time.h>

float knapsack_greedy(int p[], int w[], int m, int n) {
    float x[n], max_profit = 0.0;
    for (int i = 0; i < n; i++) {
        x[i] = 0.0;
    }

    int rem_cap = m, i = 0;
    while (rem_cap > 0) {
        if (w[i] > rem_cap) break;
        x[i] = 1.0;
        rem_cap = rem_cap - w[i];
        i++;
    }

    if (rem_cap != 0 && i < n) x[i] = (float)rem_cap / w[i];

    for (int i = 0; i < n; i++) {
        max_profit = max_profit + (x[i] * p[i]);
    }
    return max_profit;
}

void main() {
    int n, m;
    clock_t start, end;
    double time_taken;

    printf("Enter number of objects: ");
    scanf("%d", &n);
    printf("Enter max capacity of Knapsack: ");
    scanf("%d", &m);

    int p[n], w[n];
    float wp[n], profit;

    printf("\nEnter profit, weight for %d objects\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &p[i], &w[i]);
        wp[i] = (float)p[i] / w[i];
    }
}
```

```

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (wp[i] < wp[j]) {

            float temp = wp[i];
            wp[i] = wp[j];
            wp[j] = temp;

            int temp2 = w[i];
            w[i] = w[j];
            w[j] = temp2;

            int temp3 = p[i];
            p[i] = p[j];
            p[j] = temp3;
        }
    }
}

start = clock();
profit = knapsack_greedy(p, w, m, n);
end = clock();

printf("\nMaximum profit: %f", profit);

time_taken = (((double) (end - start)) / CLOCKS_PER_SEC) * 1000;
printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:

```

Enter number of objects: 3
Enter max capacity of Knapsack: 20

Enter profit, weight for 3 objects
25 18
24 15
15 10

Maximum profit: 31.500000

Time Taken: 0.003000 ms

```

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include <stdio.h>
#include <time.h>

void main() {
    int c[100][100], s[100], dist[100], v, n, u;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost between each vertex pair: \n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &c[i][j]);
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &v);

    for(int i = 0; i < n; i++) {
        s[i] = 0;
        dist[i] = c[v][i];
    }
    s[v] = 1;
    dist[v] = 0;

    for(int w = 0; w < n - 1; w++) {
        int min = 999;
        for(int i = 0; i < n; i++) {
            if(s[i] == 0 && dist[i] < min) {
                min = dist[i];
                u = i;
            }
        }
        s[u] = 1;
        for(int k = 0; k < n; k++) {
            if(s[k] == 0 && dist[k] > dist[u] + c[u][k]) {
                dist[k] = dist[u] + c[u][k];
            }
        }
    }

    for(int i = 0; i < n; i++) {
        printf("dist [%d] = %d\n", i, dist[i]);
    }
}
```

Output:

```
Enter the number of vertices: 4
Enter cost between each vertex pair:
0      10      999 30
10      0       50 999
999  50      0  20
30     999    20  0
Enter source vertex: 0
dist [0] = 0
dist [1] = 10
dist [2] = 50
dist [3] = 30
```

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int x[20], n;

int placeQueen(int k, int i) {
    for(int j = 1; j <= k - 1; j++) {
        if((x[j] == i) || (abs(j - k) == abs(x[j] - i)))
            return 0;
    }
    return 1;
}

void printSolution() {
    int i, j;

    printf("\nSolution:\n");

    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            if(x[i] == j) printf(" Q ");
            else printf(" . ");
        }
        printf("\n");
    }
}

void nQueens(int n) {
    int k = 1;
    x[k] = 0;
    while(k != 0) {
        x[k] = x[k] + 1;

        while((x[k] <= n) && !placeQueen(k, x[k]))
            x[k]++;

        if(x[k] <= n) {
            if(k == n) {
                printSolution();
                return;
            }
            else {
                k++;
                x[k] = 0;
            }
        }
    }
}
```

```

        else {
            k--;
        }
    }

    printf("No Solution Exists\n");
}

void main()
{
    clock_t start, end;

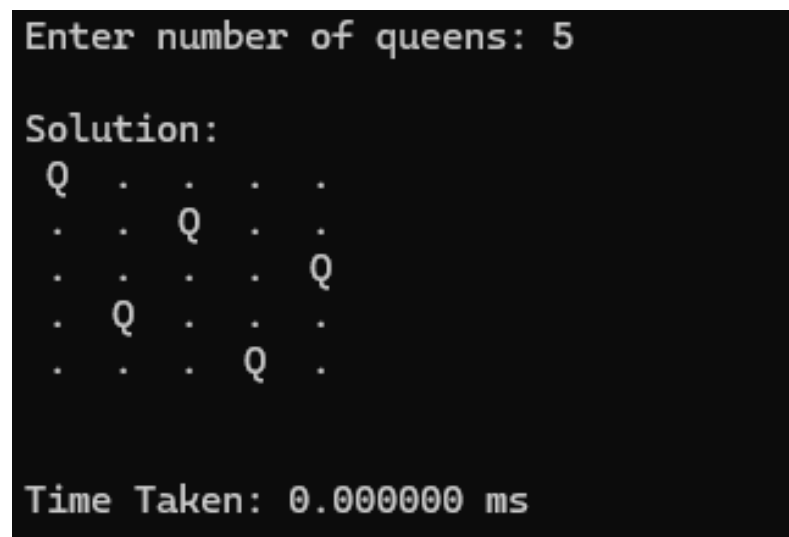
    printf("Enter number of queens: ");
    scanf("%d", &n);

    start = clock();
    nQueens(n);
    end = clock();

    double time_taken = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000;
    printf("\n\nTime Taken: %f ms\n", time_taken);
}

```

Output:



The screenshot shows the program's output on a black background with white text. It starts with the prompt "Enter number of queens: 5". Below this, it says "Solution:". The solution is displayed as a 5x5 grid where each row contains exactly one 'Q' and all other positions are empty. The 'Q's are located at (row, column) positions: (1,1), (2,3), (3,5), (4,2), and (5,4). At the bottom, it shows "Time Taken: 0.000000 ms".

```

Enter number of queens: 5

Solution:
Q   .   .   .   .
.   .   Q   .   .
.   .   .   .   Q
.   Q   .   .   .
.   .   .   Q   .

Time Taken: 0.000000 ms

```